

Ejercicio paso a paso

En este ejercicio, construiremos una aplicación Java para gestionar una tabla de **Estudiantes**. Aprenderás a conectar Java con MySQL y a realizar operaciones CRUD (Create, Read, Update, Delete).

Paso 1: Preparación de la Base de Datos

Antes de tocar el código, necesitamos una tabla en MySQL. Ejecuta este script en tu cliente SQL (MySQL Workbench, phpMyAdmin, etc.):

```
CREATE DATABASE escuela_db;
USE escuela_db;

CREATE TABLE estudiantes (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100),
    email VARCHAR(100),
    nota DECIMAL(4,2)
);
```

Paso 2: Configuración del Proyecto Java

Para que Java hable con MySQL, necesitas el conector (**Driver**).

1. Descarga el mysql-connector-j.
2. Si usas un proyecto simple, añádelo al **Build Path**. Si usas Maven, añade esta dependencia al pom.xml:

```
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>8.3.0</version>
</dependency>
```

Paso 3: Consultar Datos con Statement (SELECT)

La clase Statement se usa para sentencias SQL estáticas.

Ejercicio: Crea una clase llamada ConsultaEstatica y añade este código. Fíjate cómo usamos el bloque **try-catch-finally** (la forma antigua) para cerrar los recursos.

```
import java.sql.*;

public class ConsultaEstatica {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/escuela_db";
        String user = "root";
        String pass = "tu_password";

        Connection con = null;
        Statement stmt = null;
        ResultSet rs = null;
```

```
try {
    con = DriverManager.getConnection(url, user, pass);
    stmt = con.createStatement();
    rs = stmt.executeQuery("SELECT * FROM estudiantes");

    while (rs.next()) {
        System.out.println(rs.getString("nombre") + " - " +
rs.getDouble("nota"));
    }
} catch (SQLException e) {
    e.printStackTrace();
} finally {
    // Cerramos manualmente (tedioso y propenso a errores)
    try { if (rs != null) rs.close(); } catch (SQLException e) {}
    try { if (stmt != null) stmt.close(); } catch (SQLException e) {}
    try { if (con != null) con.close(); } catch (SQLException e) {}
}
}
```

Paso 4: Operaciones Seguras con PreparedStatement (INSERT, UPDATE, DELETE)

El PreparedStatement pre-compila la consulta y usa parámetros (?). Esto evita ataques de **SQL Injection** y es más eficiente.

El bloque try-with-resources

A partir de Java 7, ya no necesitamos el bloque finally. Al declarar los recursos dentro del try(), Java los cierra automáticamente al terminar.

Ejercicio: Crea la clase GestionEstudiantes para insertar y actualizar.

```
public class GestionEstudiantes {
    private static final String URL =
"jdbc:mysql://localhost:3306/escuela_db";
    private static final String USER = "root";
    private static final String PASS = "tu_password";

    public static void main(String[] args) {
        // INSERTAR con try-with-resources
        String sqlInsert = "INSERT INTO estudiantes (nombre, email, nota)
VALUES (?, ?, ?)";

        try (Connection con = DriverManager.getConnection(URL, USER, PASS);
            PreparedStatement pstmt = con.prepareStatement(sqlInsert)) {

            pstmt.setString(1, "Juan Perez");
            pstmt.setString(2, "juan@mail.com");
            pstmt.setDouble(3, 8.5);

            int filasAfec = pstmt.executeUpdate();
            System.out.println("Filas insertadas: " + filasAfec);

        } catch (SQLException e) {
```

Ejercicio paso a paso

```
e.printStackTrace();
}

// EJERCICIO: Crea aquí un UPDATE similar
// usando "UPDATE estudiantes SET nota = ? WHERE id = ?"
}
}
```

Paso 5: Comparativa Crítica

Es vital entender cuándo usar cada uno. Aquí tienes una tabla para tu resumen:

Característica	Statement	PreparedStatement
Uso principal	Consultas SQL fijas y simples.	Consultas con parámetros variables.
Seguridad	Vulnerable a SQL Injection.	Seguro, limpia los parámetros.
Rendimiento	Se compila cada vez que se ejecuta.	Se compila una vez, se ejecuta muchas.
Legibilidad	Difícil de leer con muchas concatenaciones.	Muy limpio usando el símbolo ?.

Paso 6: Resumen de Buenas Prácticas para el Alumno

Para que tu código sea profesional, sigue estas reglas:

1. **Usa siempre try-with-resources:** Evita fugas de memoria (memory leaks) al dejar conexiones abiertas.
2. **Prefiere PreparedStatement:** Incluso si no tienes parámetros, es una buena práctica por seguridad y hábito.
3. **Nunca guardes contraseñas en el código:** En proyectos reales, usa variables de entorno o archivos de configuración.
4. **Cierra el ResultSet:** Siempre es el primer recurso que debe cerrarse (o incluirse en el try-with-resources).

Ejercicio

Completa el programa creando un método `eliminarEstudiante(int id)` que use un `PreparedStatement` para borrar un registro por su ID y verifique si se borró algo usando el valor devuelto por `executeUpdate()`.